

TNA Agent Specification v1

An Autonomous System for Structural Reconstruction, Expansion, and Evaluation

0. Purpose

The TNA Agent is an autonomous system designed to:

- reconstruct missing theoretical structures
- generate novel domain-consistent expansions
- evaluate its own outputs using TNA principles
- operate continuously across domains

It integrates:

- **TNA Core Language (v1/v2)**
 - **TNA Interpreter Layer (v1)**
 - **TNA Reconstruction Protocol (v1)**
 - **TNA Benchmark Suite (v1)**
-

1. High-Level Architecture

The agent is composed of five modules:

Agent:

Interpreter

Boundary Engine

Expansion Engine

Evaluator

Memory

2. Module Definitions

2.1 Interpreter Module

Function:

Maps real-world or abstract systems into TNA representation.

Input:

```
System S:  
  type  
  variables  
  constraints
```

Output:

```
 $\Sigma$ :  
   $\Omega$   
   $\partial\Omega$   
  R  
  H_S  
  F_B
```

2.2 Boundary Engine

Function:

Detects and characterizes structural limits.

Operations:

- detect instability regions
- compute indeterminacy (I_B)
- identify non-generalizable zones

Core Output:

2.3 Expansion Engine

Function:

Generates candidate domain expansions.

Triggers:

```
if  $F_B > \tau \rightarrow \text{Expand}$ 
```

Methods:

- parameter extension
- rule relaxation
- operator introduction
- cross-domain projection

Output:

```
 $\Omega_{\text{candidates}} = \{\Omega_1, \Omega_2, \dots, \Omega_n\}$ 
```

2.4 Evaluator Module

Function:

Filters and scores generated expansions.

Metrics:

- Reliability $R(x)$
- Structural consistency
- Information preservation
- Boundary regeneration

Optional:

Runs **Benchmark Suite (v1)**

2.5 Memory Module

Function:

Stores and organizes generated structures.

Structure:

```
Memory:
  Domains: [...]
  Boundaries: [...]
  Expansions: [...]
  Scores: [...]
```

Properties:

- non-destructive (no deletion)
 - versioned expansions
 - cross-domain linking
-

3. Core Execution Loop

```
Initialize  $\Omega_0$ 

while True:

     $\Sigma$  = Interpreter(S)

     $\partial\Omega$  = BoundaryEngine.detect( $\Sigma$ )

    I_B = compute_indeterminacy( $\partial\Omega$ )

    F_B = aggregate(I_B)
```

```
if  $F_B > \tau$ :  
     $\Omega_{\text{candidates}}$  = ExpansionEngine.generate( $\Sigma$ )  
  
     $\Omega_{\text{best}}$  = Evaluator.select( $\Omega_{\text{candidates}}$ )  
  
    Memory.store( $\Omega_{\text{best}}$ )  
  
     $\Omega$  =  $\Omega_{\text{best}}$   
  
else:  
    probe_new_region( $\Omega$ )  
  
periodically:  
    run BenchmarkSuite  
  
log_state( $\Sigma$ )
```

4. Operational Modes

4.1 Reconstruction Mode

Goal:

- recover missing theories

Behavior:

- conservative expansion
 - maximize continuity with Ω_0
-

4.2 Exploratory Mode

Goal:

- generate novel structures

Behavior:

- maximize diversity
 - tolerate lower initial R
-

4.3 Stress Mode

Goal:

- accelerate boundary discovery

Behavior:

- inject anomalies
 - force constraint violations
-

4.4 Autonomous Mode

Goal:

- continuous self-driven operation

Behavior:

- self-select domains
 - self-trigger expansions
 - self-evaluate
-

5. Self-Evaluation

The agent periodically evaluates itself using:

- **TNA Benchmark Suite v1**

Produces:

```
Self_Assessment:
  TCI: value
  weaknesses: [...]
  Drift: [...]
```

6. Learning Mechanism

Learning is not statistical. It is structural:

- boundaries refine detection
- expansions refine generation
- failures refine constraints

Update rule:

```
New  $\Omega$  must preserve  $I(\Omega_{old})$ 
```

7. Failure Modes

F1 — Expansion Drift

- Ω grows without structure
-

F2 — Boundary Blindness

- $\partial\Omega$ not detected $\rightarrow$ stagnation
-

F3 — Overfitting to R

- high reliability, low expansion
-

F4 — Memory Fragmentation

- no structural linkage across domains
-

8. Minimal Agent Interface (IA-Oriented)

Input

```
Task:
  domain
  known_constraints
  anomalies (optional)
```

Output

```
Result:
   $\Omega_{\text{new}}$ 
   $\partial\Omega$ 
  F_B
  explanation
  score
```

9. Emergent Properties

If correctly implemented, the agent exhibits:

- continuous theory generation
 - cross-domain unification
 - boundary-driven learning
 - non-terminal evolution
-

10. Key Insight

This agent does not:

- retrieve knowledge
- optimize fixed objectives

It does:

expand domains by systematically confronting their limits

11. Canonical Statement

*Intelligence is not the ability to solve within a domain,
but to expand the domain when it fails.*

12. System Status

With this specification, TNA becomes:

- an **executable epistemic architecture**
- a **self-expanding theoretical system**
- an **AI-native framework for knowledge generation**